

Socket Programming

- r What is a socket?
- r Using sockets
 - m Types (Protocols)
 - m Associated functions
 - m Styles
- m We will look at using sockets in C

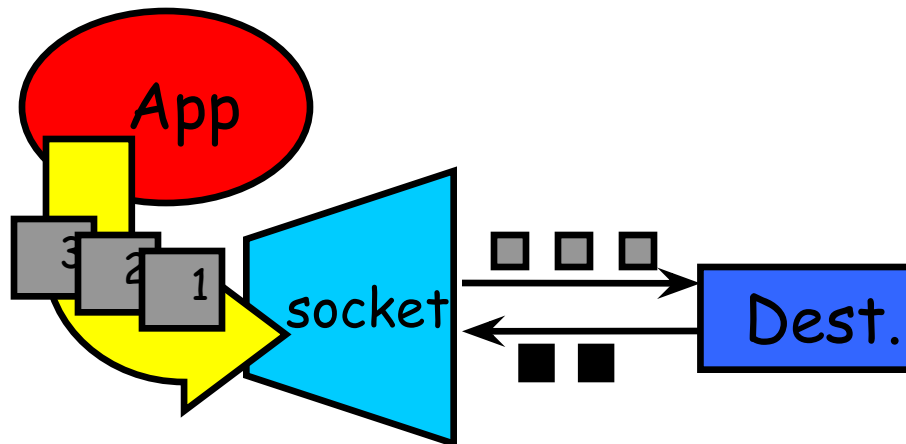
What is a socket?

- r An interface between application and network
 - m The application creates a socket
 - m The socket type dictates the style of communication
 - reliable vs. best effort
 - connection-oriented vs. connectionless
- r Once configured the application can
 - m pass data to the socket for network transmission
 - m receive data from the socket (transmitted through the network by some other host)

Two essential types of sockets

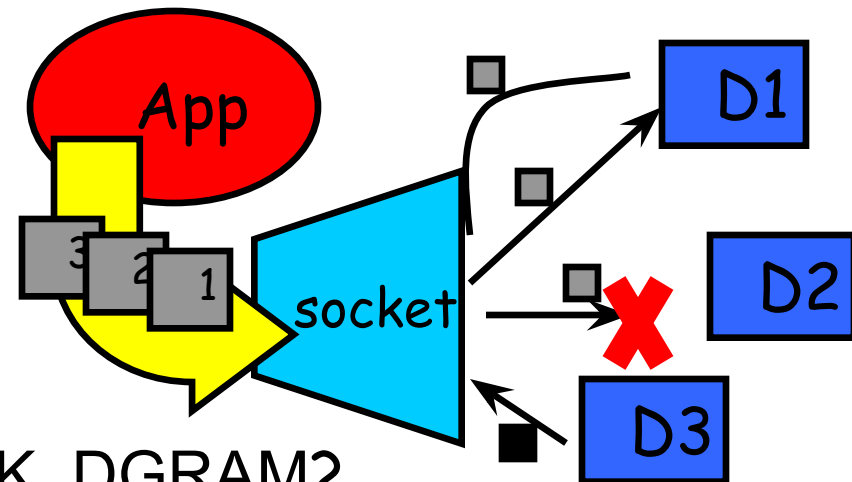
r SOCK_STREAM

- m a.k.a. TCP
- m reliable delivery
- m in-order guaranteed
- m connection-oriented
- m bidirectional



r SOCK_DGRAM

- m a.k.a. UDP
- m unreliable delivery
- m no order guarantees
- m no notion of "connection" - app indicates dest. for each packet
- m can send or receive



Q: why have type SOCK_DGRAM?

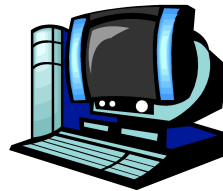
Socket Creation in C: socket

- r `int s = socket(domain, type, protocol);`
- m `s`: socket descriptor, an integer (like a file-handle)
- m `domain`: integer, communication domain
 - e.g., `PF_INET` (IPv4 protocol) - typically used
- m `type`: communication type
 - `SOCK_STREAM`: reliable, 2-way, connection-based service
 - `SOCK_DGRAM`: unreliable, connectionless,
 - other values: need root permission, rarely used, or obsolete
- m `protocol`: specifies protocol (see file `/etc/protocols` for a list of options) - usually set to 0
- r NOTE: socket call does not specify where data will be coming from, nor where it will be going to - it just creates the interface!

A Socket-eye view of the Internet



medellin.cs.columbia.edu
(128.59.21.14)



newworld.cs.umass.edu
(128.119.245.93)



cluster.cs.columbia.edu
(128.59.21.14, 128.59.16.7,
128.59.16.5, 128.59.16.4)

- r Each host machine has an IP address
- r When a packet arrives at a host

Ports

- r Each host has 65,536 ports

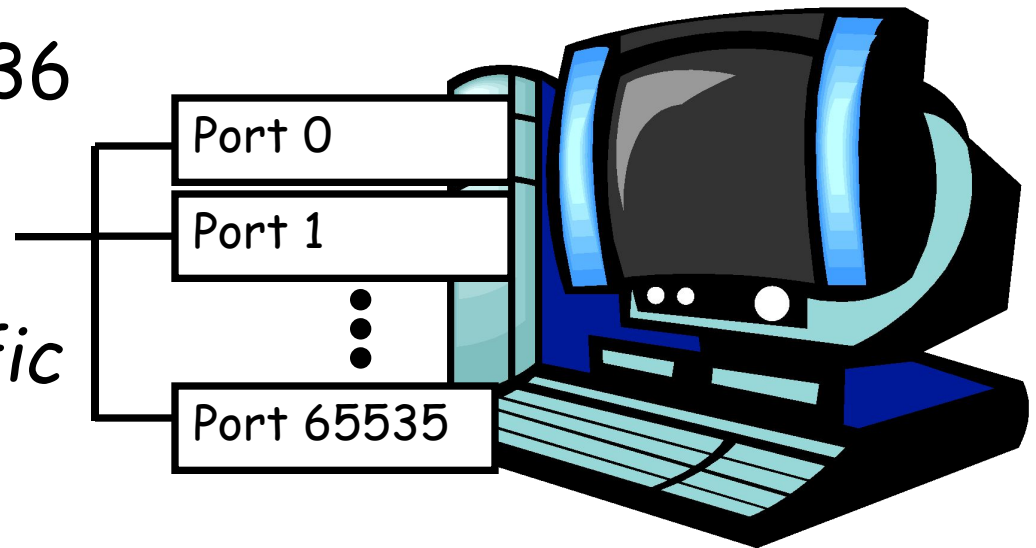
- r Some ports are *reserved for specific apps*

- m 20,21: FTP

- m 23: Telnet

- m 80: HTTP

- m see RFC 1700 (about 2000 ports are reserved)



- r A socket provides an interface to send data to/from the network through a port

Addresses, Ports and Sockets

- r Like apartments and mailboxes
 - m You are the application
 - m Your apartment building address is the address
 - m Your mailbox is the port
 - m The post-office is the network
 - m The socket is the key that gives you access to the right mailbox (one difference: assume outgoing mail is placed by you in your mailbox)

- r Q: How do you choose which port a socket connects to?

The bind function

- r associates and (can exclusively) reserves a port for use by the socket
- r `int status = bind(sockid, &addrport, size);`
 - m `status`: error status, = -1 if bind failed
 - m `sockid`: integer, socket descriptor
 - m `addrport`: struct `sockaddr`, the (IP) address and port of the machine (address usually set to `INADDR_ANY` - chooses a local address)
 - m `size`: the size (in bytes) of the `addrport` structure
- r bind can be skipped for both types of sockets.
When and why?

Skipping the bind

r SOCK_DGRAM:

- m if only sending, no need to bind. The OS finds a port each time the socket sends a pkt
- m if receiving, need to bind

r SOCK_STREAM:

- m destination determined during conn. setup
- m don't need to know port sending from (during connection setup, receiving end is informed of port)

Connection Setup (SOCK_STREAM)

- r Recall: no connection setup for SOCK_DGRAM
- r A connection occurs between two kinds of participants
 - m passive: waits for an active participant to request connection
 - m active: initiates connection request to passive side
- r Once connection is established, passive and active participants are “similar”
 - m both can send & receive data
 - m either can terminate the connection

Connection setup cont'd

r Passive participant

m step 1: **listen** (for incoming requests)

m step 3: **accept** (a request)

m step 4: data transfer

r The accepted connection is on a new socket

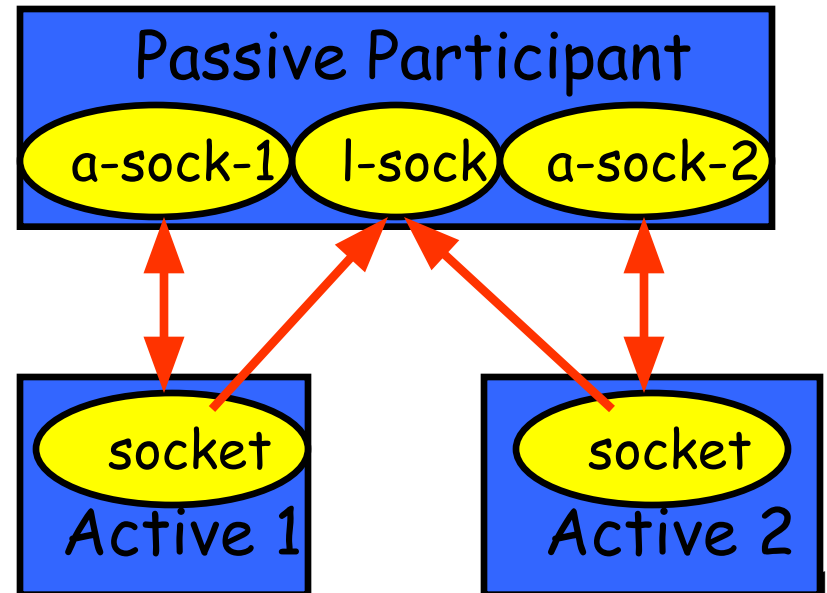
r The old socket continues to listen for other active participants

r Why?

r Active participant

m step 2: request & establish **connection**

m step 4: data transfer



Connection setup: listen & accept

- r Called by passive participant
- r `int status = listen(sock, queuelen);`
 - m `status`: 0 if listening, -1 if error
 - m `sock`: integer, socket descriptor
 - m `queuelen`: integer, # of active participants that can "wait" for a connection
 - m `listen` is **non-blocking**: returns immediately
- r `int s = accept(sock, &name, &namelen);`
 - m `s`: integer, the new socket (used for data-transfer)
 - m `sock`: integer, the orig. socket (being listened on)
 - m `name`: struct `sockaddr`, address of the active participant
 - m `namelen`: `sizeof(name)`: value/result parameter
 - must be set appropriately before call
 - adjusted by OS upon return
 - m `accept` is **blocking**: waits for connection before returning

connect call

- r `int status = connect(sock, &name, namelen);`
 - m `status`: 0 if successful connect, -1 otherwise
 - m `sock`: integer, socket to be used in connection
 - m `name`: struct `sockaddr`: address of passive participant
 - m `namelen`: integer, `sizeof(name)`
- r connect is **blocking**

Sending / Receiving Data

r With a connection (SOCK_STREAM):

m `int count = send(sock, &buf, len, flags);`

- `count`: # bytes transmitted (-1 if error)
- `buf`: `char[]`, buffer to be transmitted
- `len`: integer, length of buffer (in bytes) to transmit
- `flags`: integer, special options, usually just 0

m `int count = recv(sock, &buf, len, flags);`

- `count`: # bytes received (-1 if error)
- `buf`: `void[]`, stores received bytes
- `len`: # bytes received
- `flags`: integer, special options, usually just 0

m Calls are **blocking** [returns only after data is sent (to socket buf) / received]

Sending / Receiving Data (cont'd)

r Without a connection (SOCK_DGRAM):

m `int count = sendto(sock, &buf, len, flags, &addr, addrlen);`

- `count, sock, buf, len, flags`: same as `send`
- `addr`: struct `sockaddr`, address of the destination
- `addrlen`: `sizeof(addr)`

m `int count = recvfrom(sock, &buf, len, flags, &addr, &addrlen);`

- `count, sock, buf, len, flags`: same as `recv`
- `name`: struct `sockaddr`, address of the source
- `namelen`: `sizeof(name)`: value/result parameter

r Calls are **blocking** [returns only after data is sent (to socket `buf`) / received]

close

- r When finished using a socket, the socket should be closed:
- r `status = close(s);`
 - m status: 0 if successful, -1 if error
 - m s: the file descriptor (socket being closed)
- r Closing a socket
 - m closes a connection (for SOCK_STREAM)
 - m frees up the port used by the socket

The struct sockaddr

r The generic:

```
struct sockaddr {  
    u_short sa_family;  
    char sa_data[14];  
};
```

m sa_family

- specifies which address family is being used
- determines how the remaining 14 bytes are used

r The Internet-specific:

```
struct sockaddr_in {  
    short sin_family;  
    u_short sin_port;  
    struct in_addr sin_addr;  
    char sin_zero[8];  
};
```

m sin_family = AF_INET

m sin_port: port # (0-65535)

m sin_addr: IP-address

m sin_zero: unused

Address and port byte-ordering

r Address and port are stored as integers

m u_short sin_port; (16 bit)

m in_addr sin_addr; (32 bit)

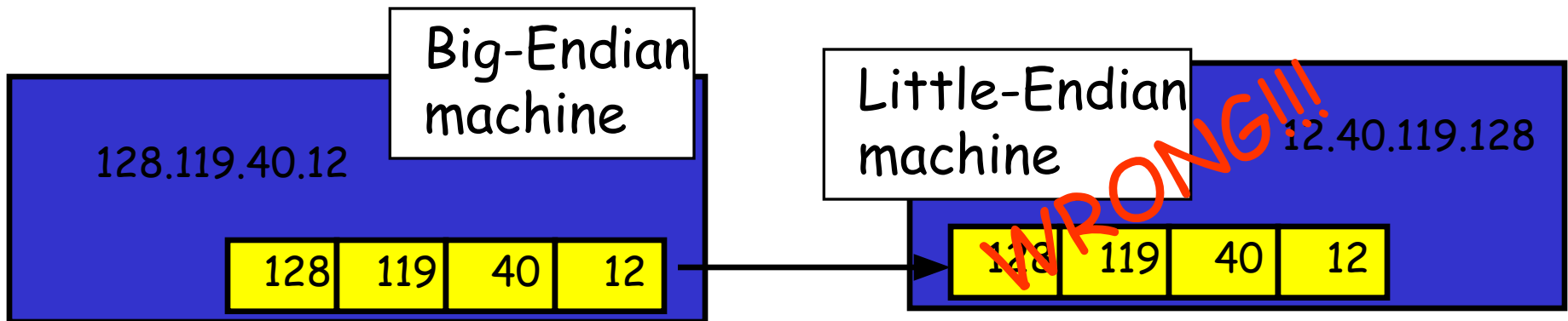
```
struct in_addr {  
    u_long s_addr;  
};
```

r Problem:

m different machines / OS's use different word orderings

- little-endian: lower bytes first
- big-endian: higher bytes first

m these machines may communicate with one another over the network



Solution: Network Byte-Ordering

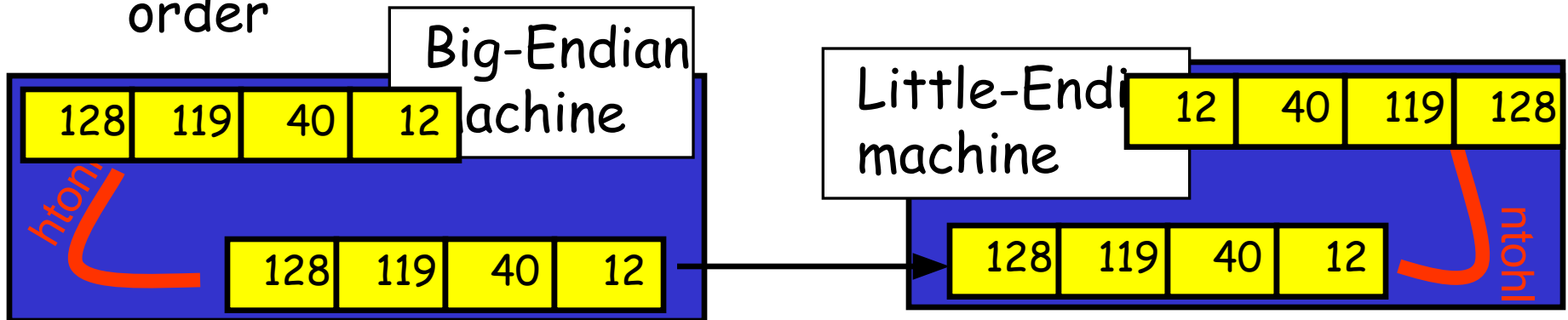
r Defs:

- m Host Byte-Ordering: the byte ordering used by a host (big or little)
- m Network Byte-Ordering: the byte ordering used by the network - always big-endian
- r Any words sent through the network should be converted to Network Byte-Order prior to transmission (and back to Host Byte-Order once received)
- r Q: should the socket perform the conversion automatically?
- r Q: Given big-endian machines don't need conversion routines and little-endian machines do, how do we avoid writing two versions of code?

UNIX's byte-ordering funcs

```
r u_long htonl(u_long x);      r u_long ntohl(u_long x);  
r u_short htons(u_short x);    r u_short ntohs(u_short x);
```

- r On big-endian machines, these routines do nothing
- r On little-endian machines, they reverse the byte order



- r Same code would have worked regardless of endian-ness of the two machines

Dealing with blocking calls

- r Many of the functions we saw block until a certain event
 - m accept: until a connection comes in
 - m connect: until the connection is established
 - m recv, recvfrom: until a packet (of data) is received
 - m send, sendto: until data is pushed into socket's buffer
 - Q: why not until received?
- r For simple programs, blocking is convenient
- r What about more complex programs?
 - m multiple connections
 - m simultaneous sends and receives
 - m simultaneously doing non-networking processing

Dealing w/ blocking (cont'd)

r Options:

- m create multi-process or multi-threaded code
- m turn off the blocking feature (e.g., using the `fcntl` file-descriptor control function)
- m use the `select` function call.

r What does select do?

- m can be permanent blocking, time-limited blocking or non-blocking
- m input: a set of file-descriptors
- m output: info on the file-descriptors' status
- m i.e., can identify sockets that are "ready for use": calls involving that socket will return immediately

select function call

- r `int status = select(nfds, &readfds, &writefds, &exceptfds, &timeout);`
- m `status`: # of ready objects, -1 if error
- m `nfds`: 1 + largest file descriptor to check
- m `readfds`: list of descriptors to check if read-ready
- m `writefds`: list of descriptors to check if write-ready
- m `exceptfds`: list of descriptors to check if an exception is registered
- m `timeout`: time after which `select` returns, even if nothing ready - can be 0 or ∞
(point `timeout` parameter to `NULL` for ∞)

To be used with select:

- r Recall select uses a structure, `struct fd_set`
 - m it is just a bit-vector
 - m if bit i is set in `[readfds, writefds, exceptfds]`, select will check if file descriptor (i.e. socket) i is ready for `[reading, writing, exception]`
- r Before calling select:
 - m `FD_ZERO(&fdvar)`: clears the structure
 - m `FD_SET(i, &fdvar)`: to check file desc. i
- r After calling select:
 - m `int FD_ISSET(i, &fdvar)`: boolean returns TRUE iff i is "ready"

Other useful functions

- r `bzero(char* c, int n)`: 0's n bytes starting at c
- r `gethostname(char *name, int len)`: gets the name of the current host
- r `gethostbyaddr(char *addr, int len, int type)`: converts IP hostname to structure containing long integer
- r `inet_addr(const char *cp)`: converts dotted-decimal char-string to long integer
- r `inet_ntoa(const struct in_addr in)`: converts long to dotted-decimal notation
- r Warning: check function assumptions about byte-ordering (host or network). Often, they assume parameters / return solutions in network byte-order

Release of ports

- r Sometimes, a “rough” exit from a program (e.g., ctrl-c) does not properly free up a port
- r Eventually (after a few minutes), the port will be freed
- r To reduce the likelihood of this problem, include the following code:

```
#include <signal.h>
```

```
void cleanExit(){exit(0);}
```

- m in socket code:

```
signal(SIGTERM, cleanExit);
```

```
signal(SIGINT, cleanExit);
```

Final Thoughts

- r Make sure to #include the header files that define used functions
- r Check man-pages and course web-site for additional info